Slides: Restricted access to CS 422 Fall 2026; not to be shared online or in public forums

Slides adapted from Prof. Stefan Schmid's course at TU Berlin, and the Top-Down Approach textbook by Kurose-Ross

TCP Congestion Control

CS 422: Computer Networks

Instructor: Vamsi Addanki

TAs: Youngsuk Kim, Xiao Luo, Albert Vo

11 September 2026

PURDUE UNIVERSITY | Department of Computer Science

Reminder - TCP

- Connections have to be established and terminated
- Not stateless
- Sequence numbers to ensure the correct order between messages
- Lost messages are detected (timeout) and will be resent
  - Receiver detects missing packets through missing sequence no.
  - Repeats old ACK (sends “duplicate ACK”)
- TCP uses a sliding window
  - So receiver is not overwhelmed
  - “Flow Control” (≠ Congestion Control)

Congestion

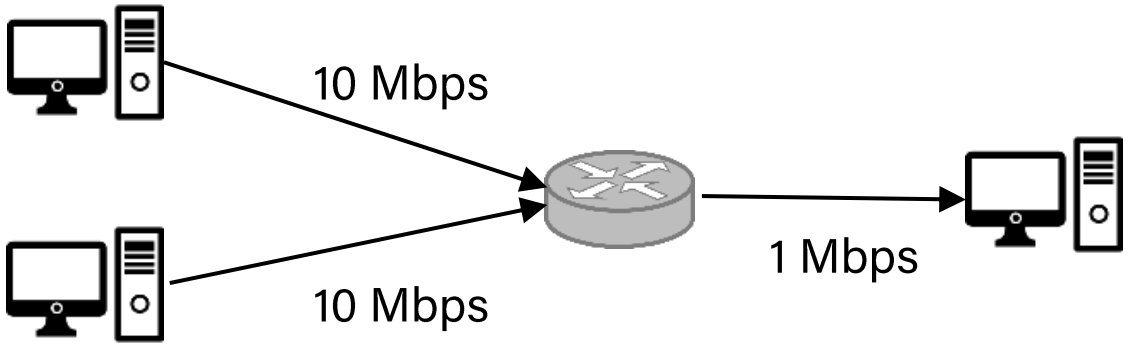
- Informally:

"Too **many sources** sending
too much data
too fast for network to handle"

- Different from flow control!
- Manifestations:
  - Lost packets (buffer overflow at routers)
  - Long delays (queueing in router buffers)
- A top-10 problem in the Internet!

Congestion

- What is the problem with this setup?

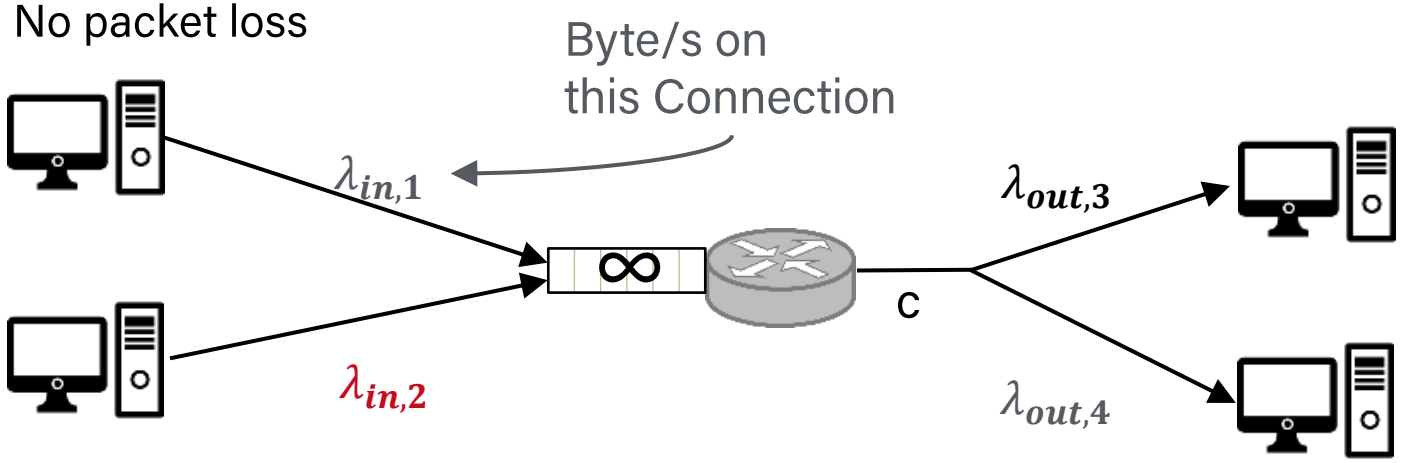


➔ Different sources compete for resources inside the network

- Sources are unaware of current state of resource
- Sources are unaware of each other
- Throughput < 1 Mbps (congestion collapse)

Cost of Congestion: Infinite Buffers

- Router with **infinite** buffer
- No packet loss



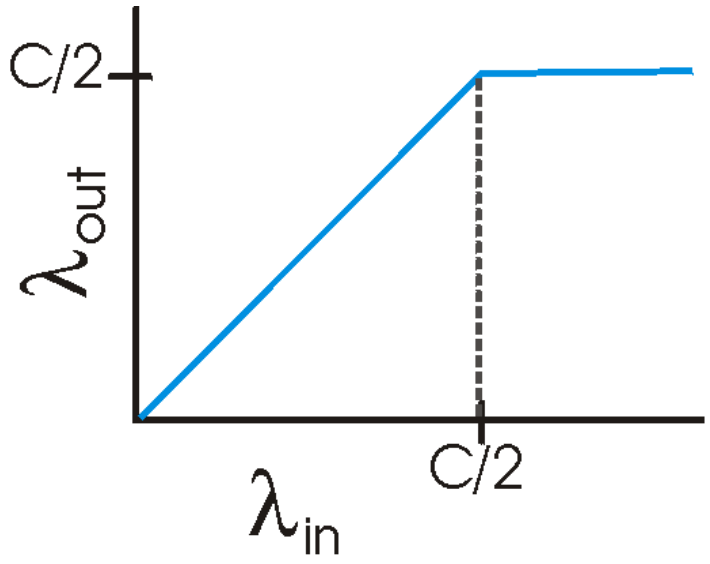
➔ Very short queue, when sending rate is equal or less than bandwidth of receivers

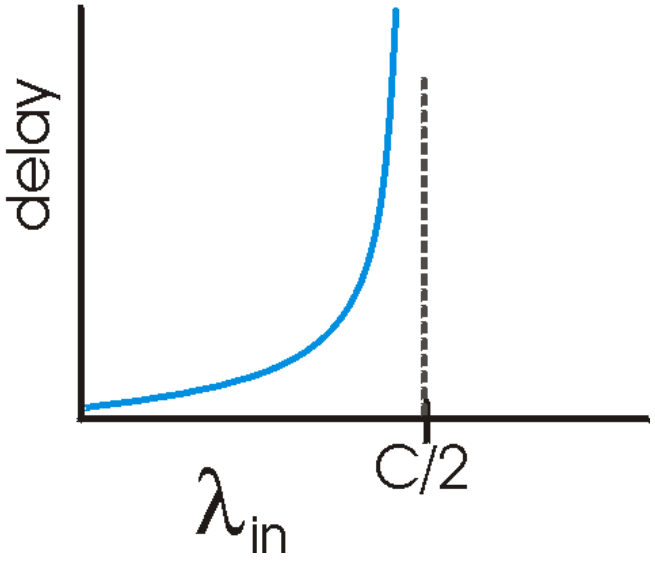
$$(\lambda_{in,1}+\lambda_{in,2}) \leq (\lambda_{out,3}+\lambda_{out,4})$$

➔ Large delays, when sending rate higher than available bandwidth of receivers

$$(\lambda_{in,1}+\lambda_{in,2}) > (\lambda_{out,3}+\lambda_{out,4})$$

Cost of Congestion: Infinite Buffers



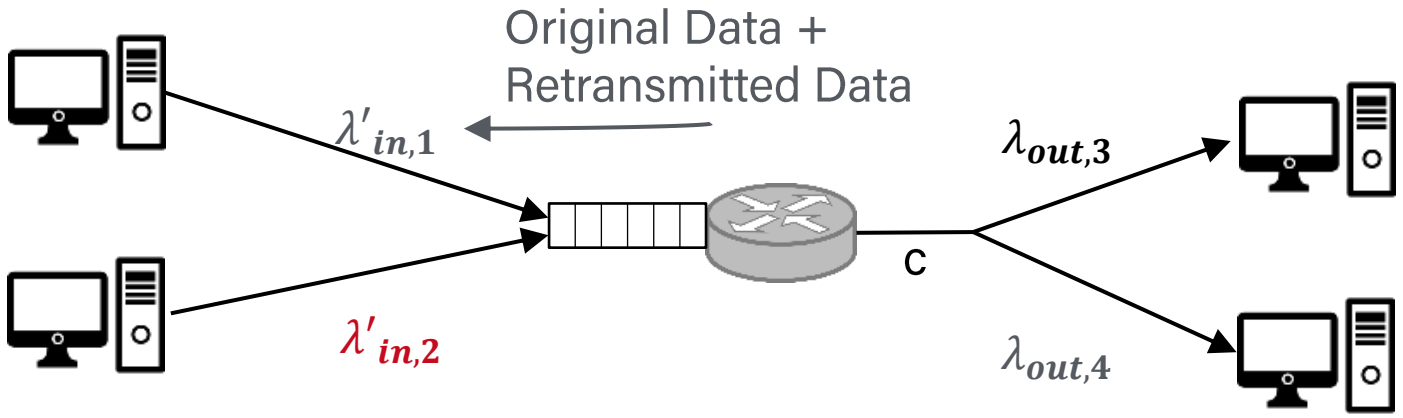


λ_{in} : Data rate of stream between one sender to the router
λ_{out} : Data rate of stream between router and receiver
C : Capacity of outgoing shared link

Cost of Congestion: Finite Buffer

- Router with finite buffer
- Retransmissions of lost packets

▶ When buffer is full: Packets will be dropped ("Drop-Tail")



➔ Retransmissions decrease goodput (throughput of application layer/ "useful" data)

➔ When packet is stuck in queue: Sender sees timeout and sends copy of packet although it is not needed

Complex Scenario with Retransmits

- What happens when traffic increases?

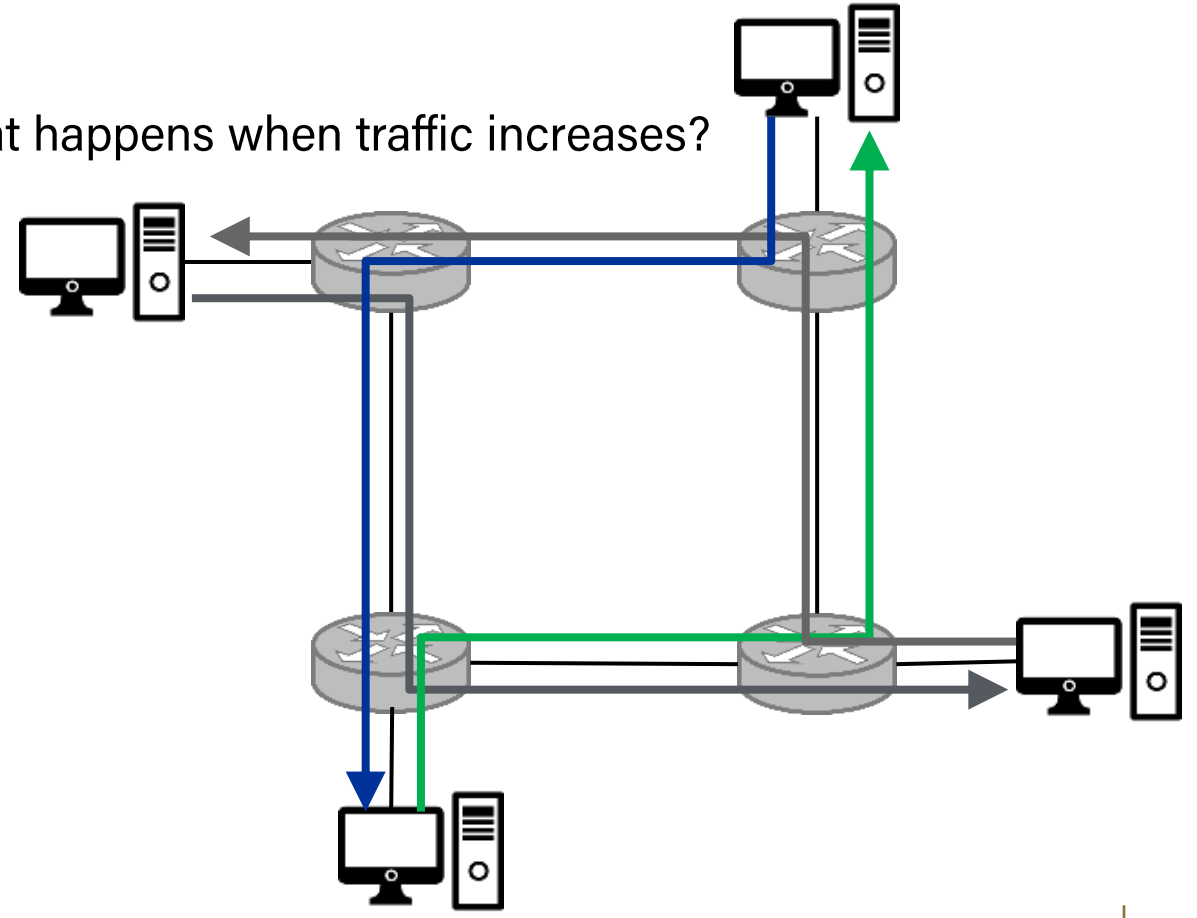


Diagram: Kurose-Ross

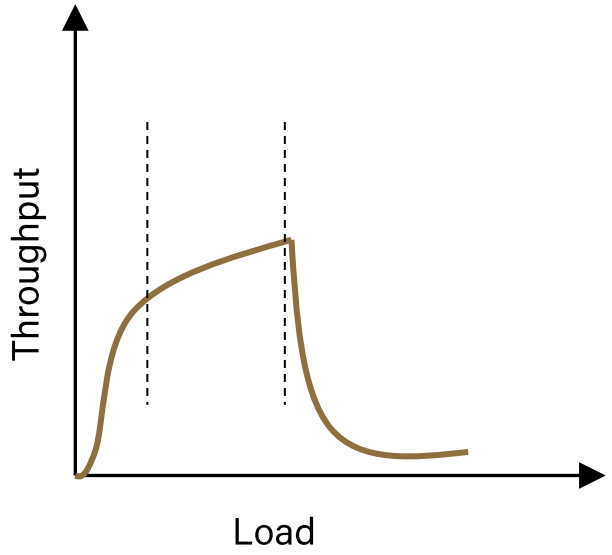
Complex Scenario with Retransmits

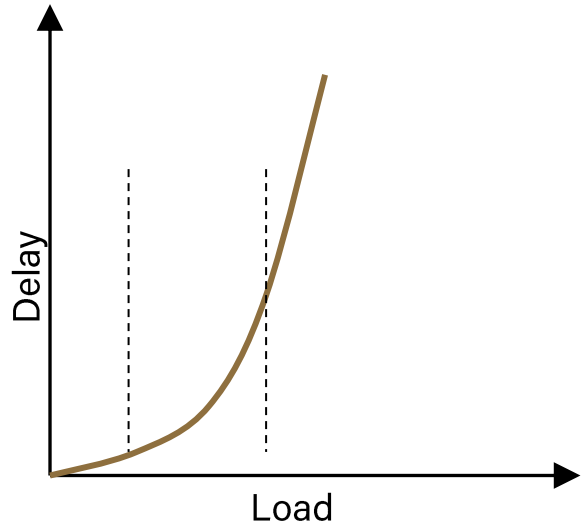
- What happens when traffic increases?
  - End-to-end throughput goes to zero
  - When packet is dropped somewhere in network, previously invested bandwidth is wasted

Congestion Collapse

- Overload cause network to collapse and no useful data can be transmitted although all systems are online
- Was a big problem in the 80s on the early Internet
  - Drop in throughput by 3 orders of magnitude
- Causes
  - Retransmissions of packets still in flight
  - Packets consume resources, but are dropped elsewhere
  - Mismatch of transmission and retransmission units
  - Large percentage of traffic is for control (e.g. headers, DNS…)
  - Stale or unwanted packets
  - …
- Congestion control ensures functionality of Internet

Congestion Collapse





Where to prevent collapse?

- At the hosts?
  - Yes, this is the common case
  - Sending host adjusts amount of data it sends
  - Sending host tries to detect congestion
- At the routers?
  - No, not all forms of collapse
  - Doesn't mean they can't help
  - Can send congestion signals
  - Isolating ill-behaved sources

Congestion Control and Avoidance

- Generally, mechanism which
  - Ensures network resources are used efficiently
  - Preserves fair network resource allocation
  - Prevents or avoids collapse
- Goals of end-to-end congestion control
  - Simple router behavior
  - Distributed-ness
  - Efficiency: $X_{knee} = \Sigma x_i(t)$
  - Fairness: $(\Sigma x_i)^2/n(\Sigma x_i^2)$
  - Power: $(throughput^{\alpha}/delay)$
  - Convergence: control system must be stable
- Avoidance keeps the system performing
- Control kicks in once the system has reached a congested state

TCP Congestion Control

- End-to-end control: Network core does not assist
- Tries to find the right window size
  - Transmit as fast as possible without loss
  - Loss is sign of congestion (not necessarily true on all link types)
  - Increase window until loss detected, then decrease window again

  ➔ Probing for usable bandwidth (continuously)

- Consists of 2 phases
  - Slow start
  - Congestion avoidance

TCP Slowstart

- Start with window of size 1 MSS (Maximum Segment Size)
- Per RTT increase window size (*cwnd*) exponentially
- Keep increasing until ...
  - Window size lager than preset threshold
  - Loss detected
- Loss event: Timeout (Tahoe TCP) and/or or three duplicate ACKs (Reno TCP)

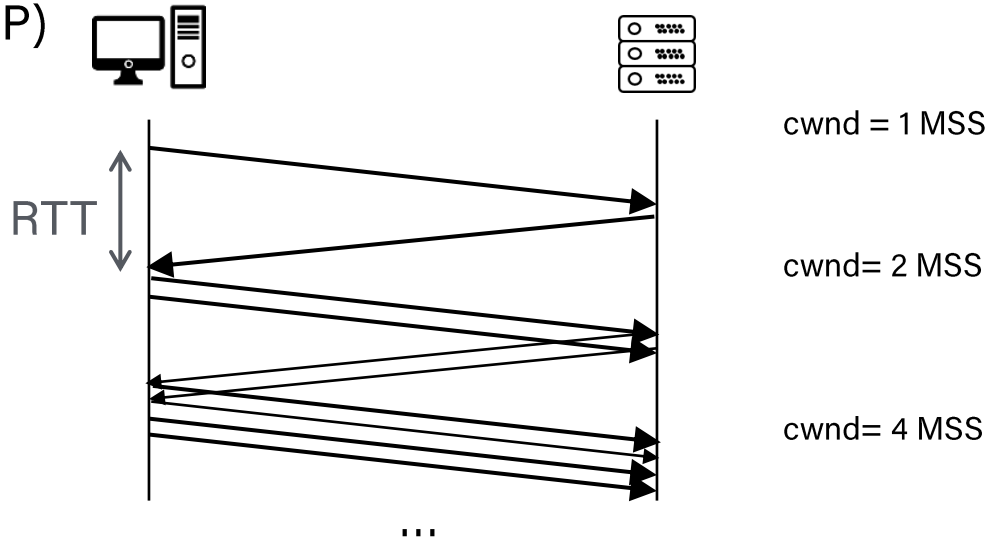


Diagram: Kurose-Ross

TCP Congestion Avoidance

- When threshold is reached: Sender switches from slowstart to congestion avoidance mode
  - Halve window size: $cwnd_{new} = \frac{1}{2} cwnd_{old}$
  - Increase window by $1 \frac{\text{MSS}}{RTT}$

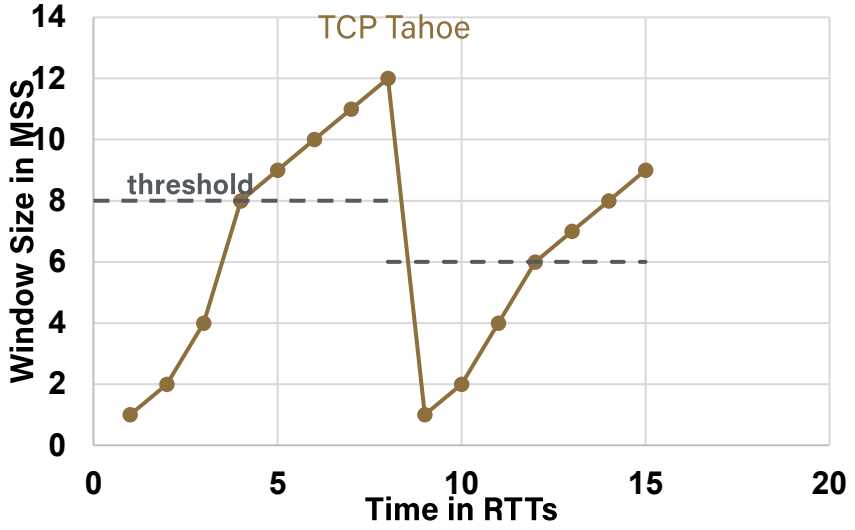


Diagram: Kurose-Ross

AIMD

- Additive increase and multiplicative decrease (**AIMD**)
  - Increase additively as long as path is congestion-free
  - Multiplicative decrease when congestion is detected

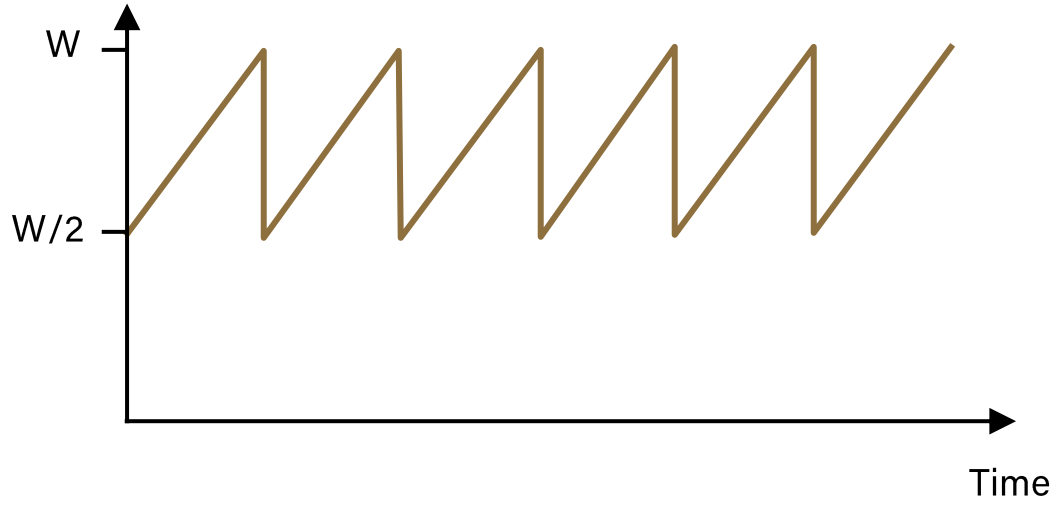
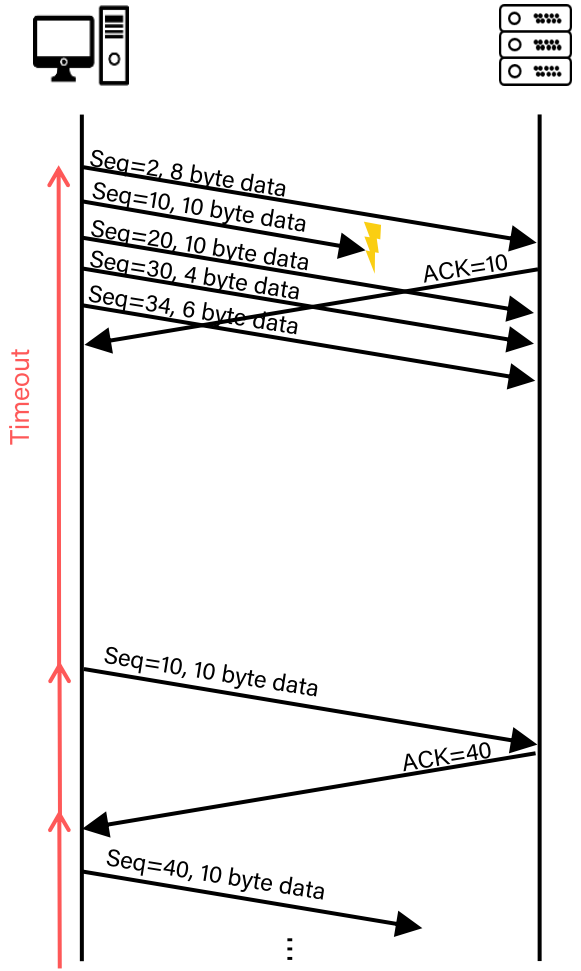


Diagram: Kurose-Ross

Fast Retransmit - Motivation

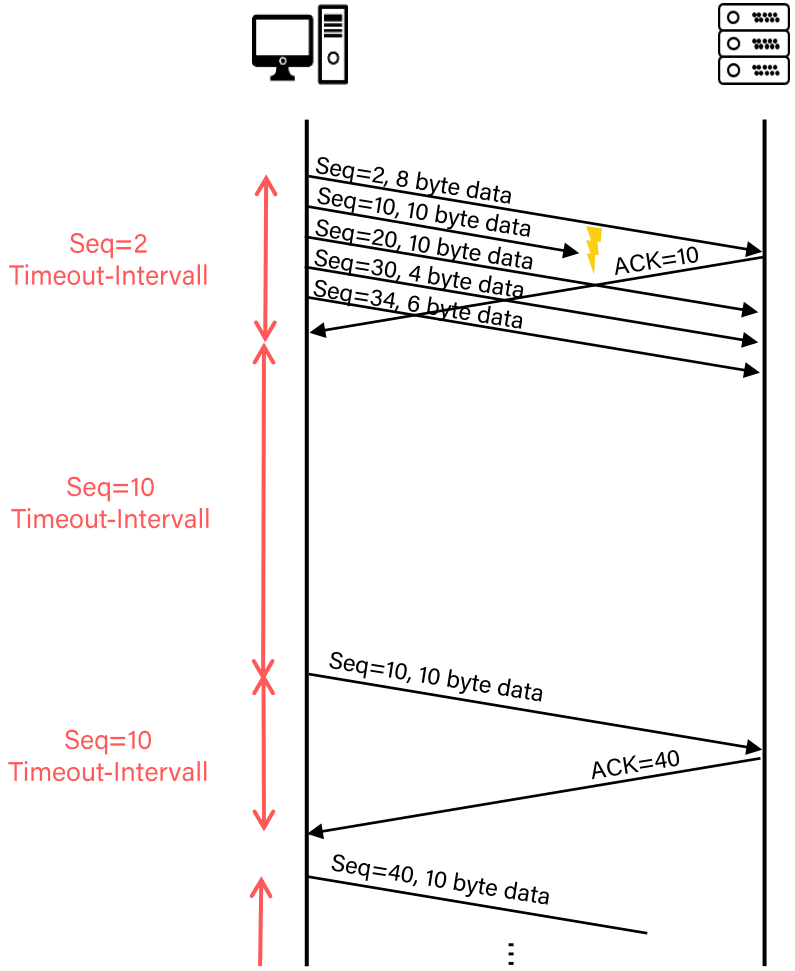
- Problem with Retransmits
  - Timeout is long
  - Retransmit of segment is delayed
  - Increased end-to-end delay
- But: receiver sees gap in data stream, since segment is missing
  - Segment lost by network
  - Reordering of segments in network (e.g. due to multiple transmission paths)
- How can receiver signal missing segments to the sender?



Fast Retransmit

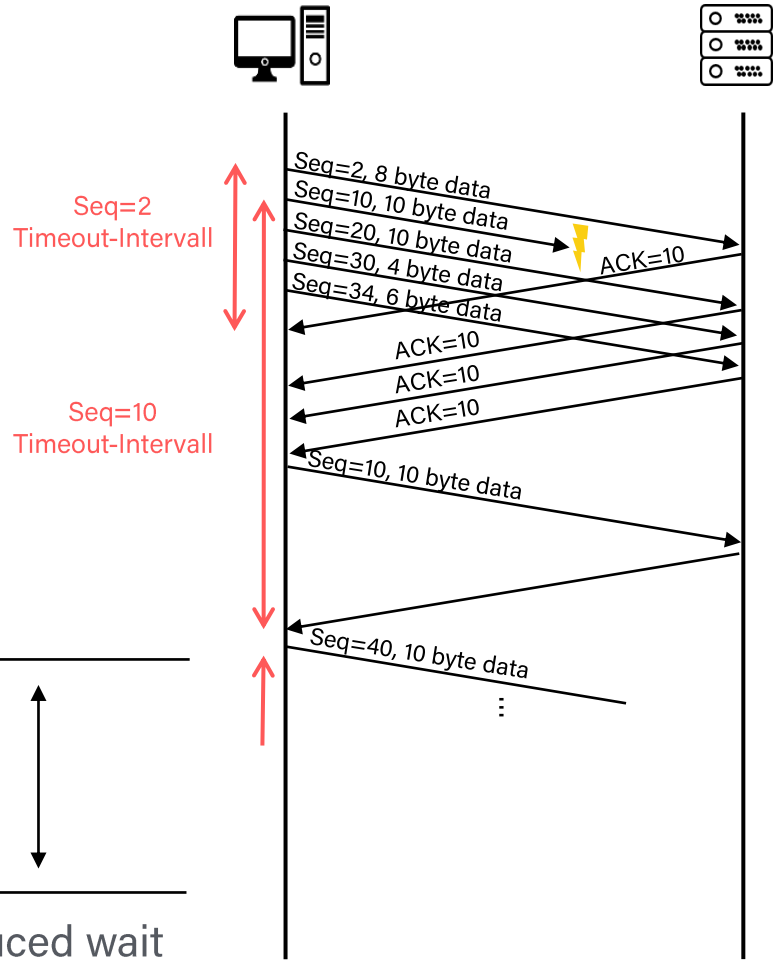
- Signal gap between segments using a duplicate ACK
  - Receiver confirms last byte that was received in-order
  - All following segments are out-of-order
- Sender usually sends many segments at the same time
  - On segment loss: many successive duplicate ACKs
  - Three duplicate ACKs (TDACK) for same byte: Indicator for packet loss
- When TDACK is received
  - Sender retransmits missing segment before timeout
  - Sender sets cwnd to 1 and transitions to **slowstart** Phase

Fast Retransmit - Comparison



TCP without Fast Retransmit

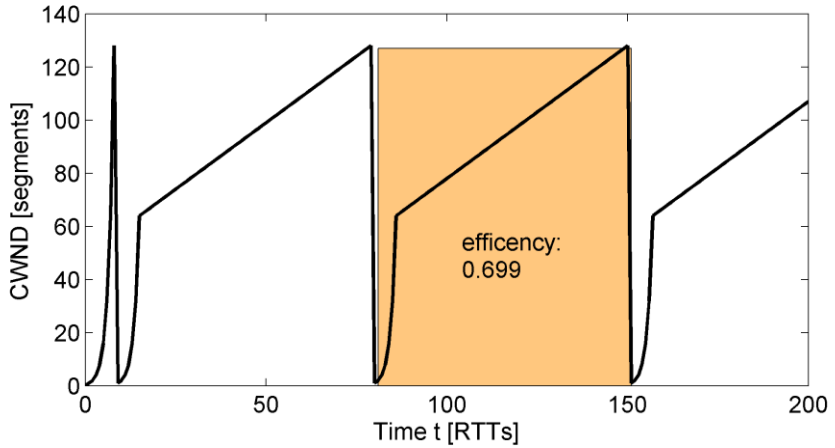
Reduced wait time due to Retransmit



TCP with Fast Retransmit

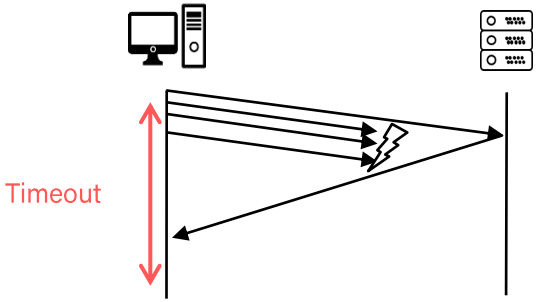
Fast Recovery - Motivation

- After gap between segments
  - Threshold and congestion window are adapted
  - Transition to slowstart
  - Result: Average bandwidth < 70% of maximum
- How to increase efficiency?

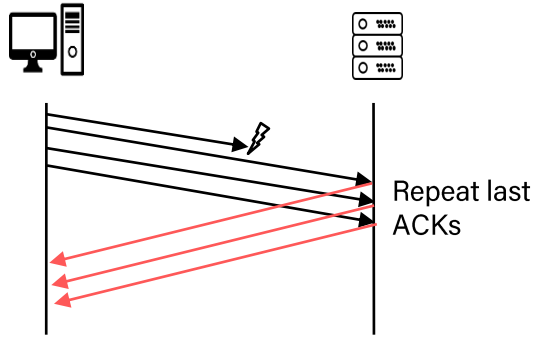


Timeouts vs. Triple Duplicate ACKs

- Timeouts
  - No other messages come thought
  - Strong sign for congestion



- Triple duplicate ACK (TDACK)
  - Other packets still came through
  - Network still capable to deliver some packets
  - Probably not congestion
  - Detecting TDACKs takes only one RTT



Fast Recovery

- React to packet loss according to detection method
- Timeout
  - Congestion detected!
  - Set $cwnd_{new} = 1$
  - Transition to slowstart mode
- TDACK
  - Only singular event
  - Adapt threshold: $threshold_{new} = \frac{1}{2} cwnd_{old}$
  - Adapt cwnd: $cwnd_{new} = threshold_{new}$
  - Transition to congestion avoidance mode
  - Kurose/Ross suggest to additionally increased the cwnd by 3 as good measure to account for the triple duplicated ACKs received before the transition to congestion avoidance mode

TCP Tahoe vs. TCP Reno

- TCP Tahoe:
  - Implements slowstart and congestion avoidance
  - When any kind of loss occurs: Restart with slowstart
- TCP Reno:
  - Implements slowstart and congestion avoidance
  - Differentiates between TDACKs and timeouts
  - When timeout: Restart with slowstart
  - When TDACK: Do a **fast retransmit**, then go into **fast recovery**

TCP Tahoe vs. TCP Reno

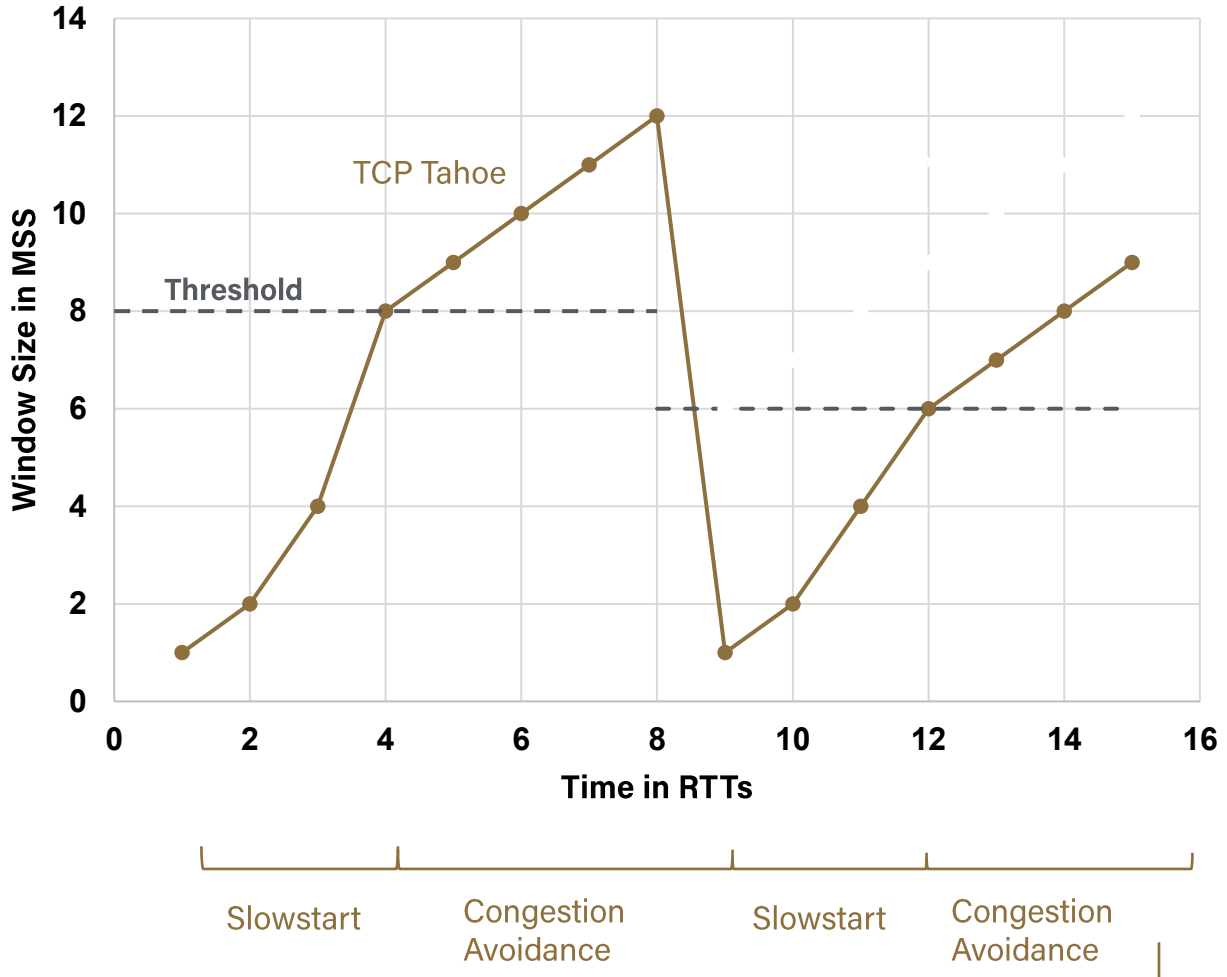
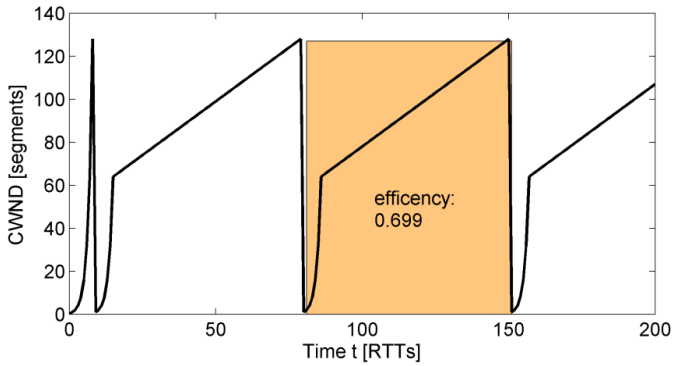


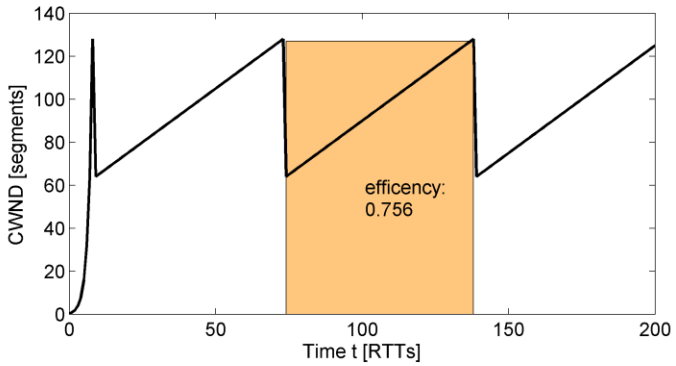
Diagram: Kurose-Ross

TCP Tahoe vs. TCP Reno

Tahoe



Reno



- Increased throughput in Reno
  - Average gain depends on probability of timeouts
  - More than 5% increase, in case no timeouts occur

TCP Extensions

- TCP New-Reno
  - Stay in fast recovery until all packet losses in window are recovered
  - In Reno: until increase in sequence numbers is detected
  - Can recover 1 packet loss per RTT without causing a timeout
- Selective Acknowledgements (SACK) [rfc2018]
  - Provides information about out-of-order packets received by receiver
  - Can recover multiple packet losses per RTT

Other TCP Versions

- Many more TCP versions exist
  - TCP Vegas
  - TCP Cubic (used in current Linux kernel)
  - …

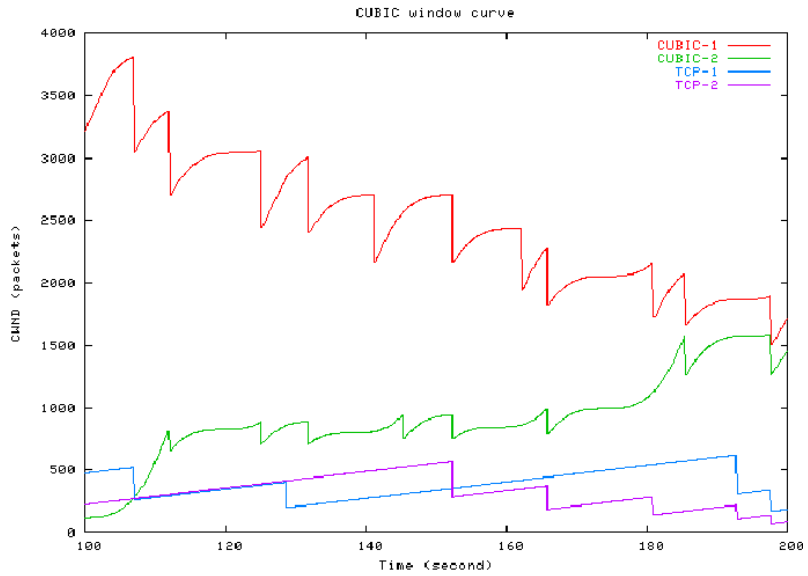
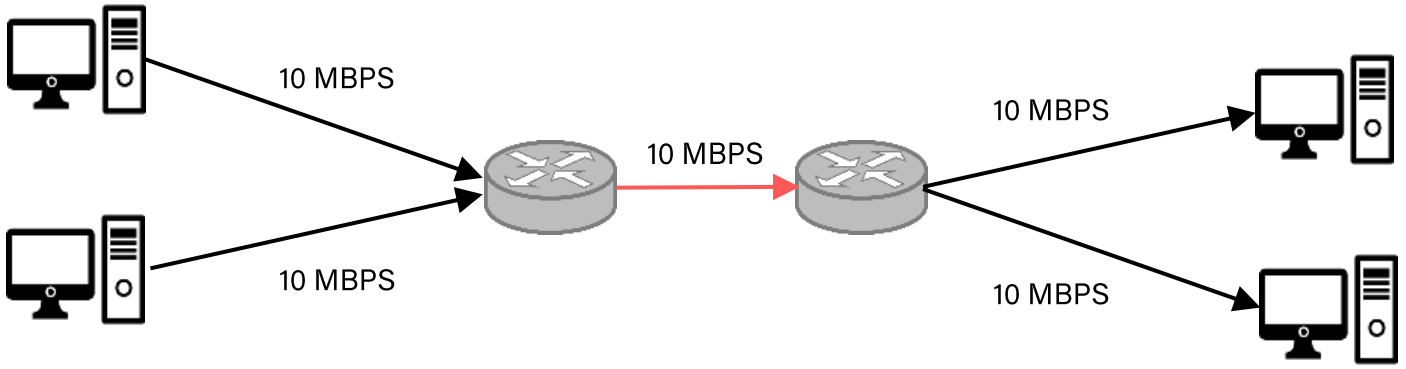


Diagram: Sangtae Ha et al.

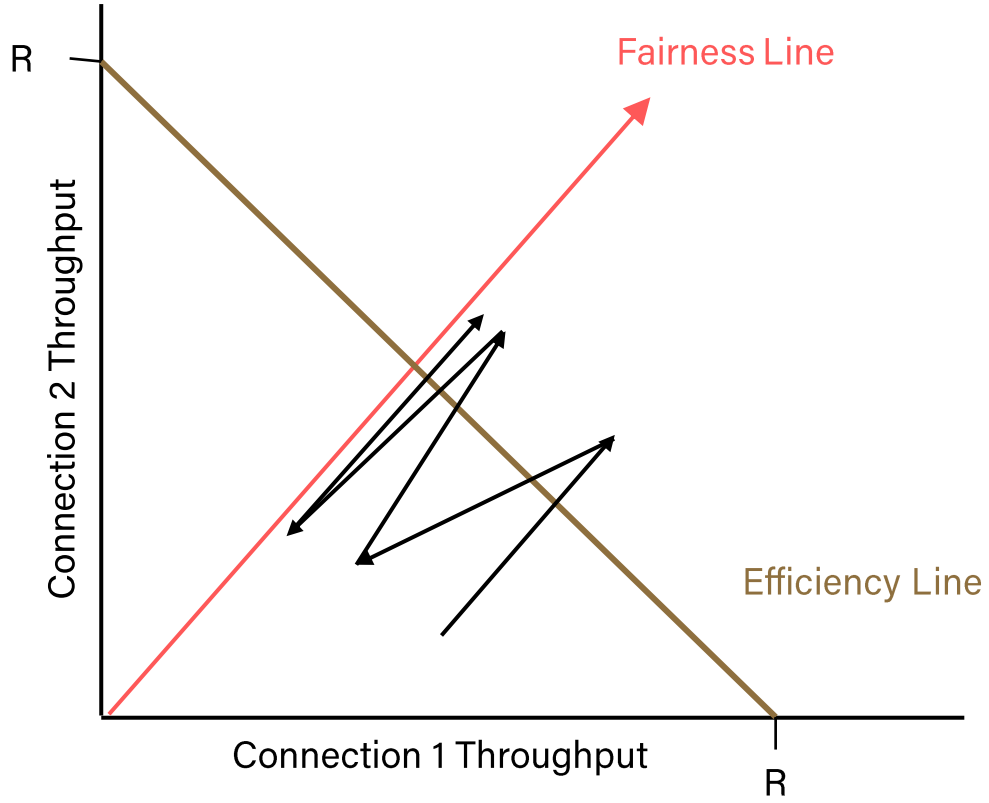
Fairness

- Fairness goal
  - *N* TCP sessions share same bottleneck link
  - Each gets *1/N* of link capacity
- Is TCP fair?



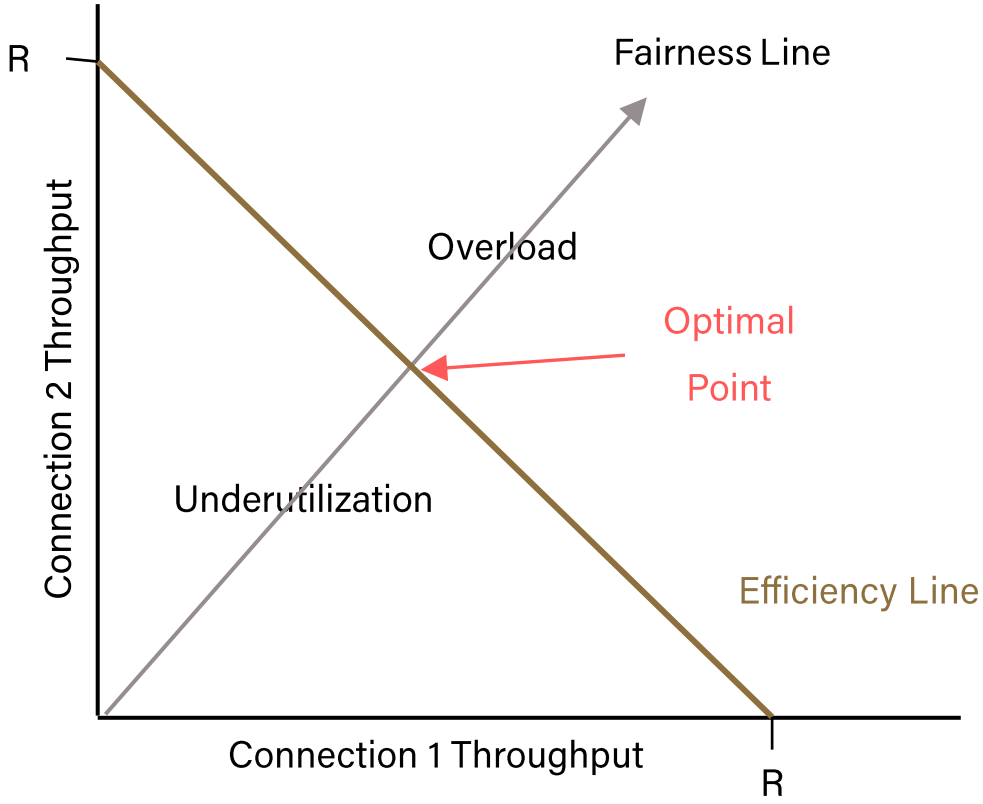
Phase Plots

- Simple way to visualize behavior of competing connections over time



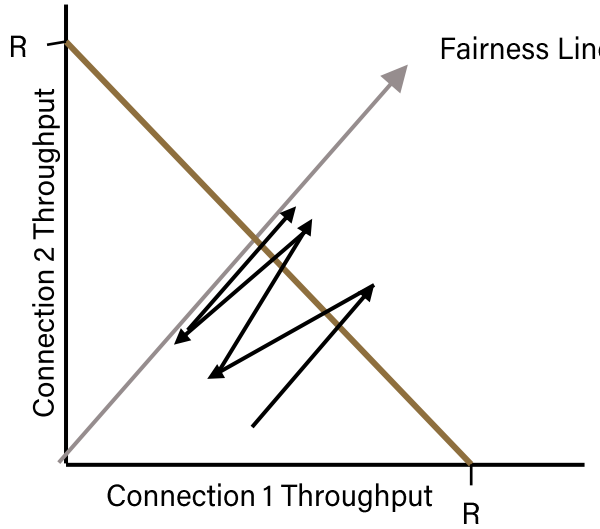
Phase Plots

- What are desirable properties?
- What if flows are not equal?



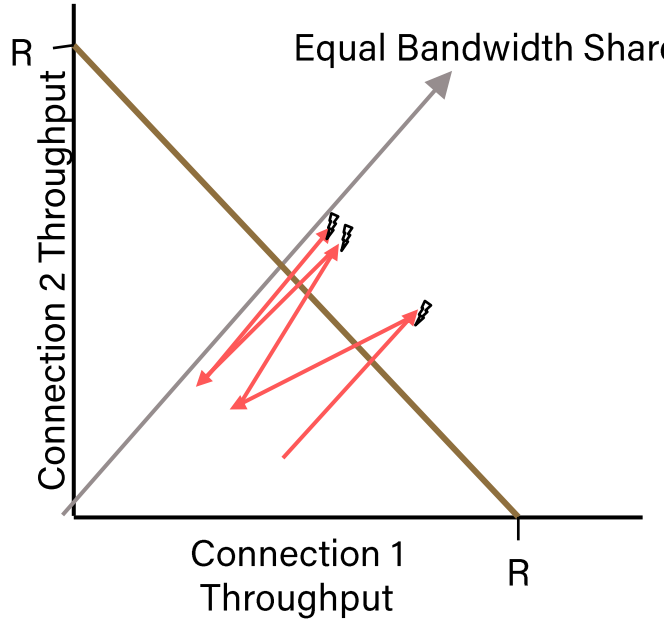
Distributed Convergence to Efficiency

- Additive increase/ decrease
  - Both senders increase/decrease by same amount over time
- Multiplicative increase/decrease
  - Both senders in-/decrease by the same factor
- Converge quickly to intersection of fairness and efficiency lines



Fairness in TCP: Ideal Case

- When one connection tries to size more than bandwidth..
  - Loss event more likely at this connection
  - Multiplicative decrease makes window (so the sending rate) significantly smaller
  - Sender then starts increasing the window again, but slowly (additively)



Assumptions on TCP Fairness

- Window under consideration is large enough
- Same TCP version
  - Different congestion behavior
  - Different parameters for additive increase: Windows grow faster or slower
- Same RTT
  - Long RTTs cause senders to react slower to congestion
  - Senders with longer RTT do not increase window as fast as senders with shorter RTT
- Similar TCP parameters
- Enough data to send
- ....

UDP Sender vs TCP Sender

- What happens when a UDP sender and a TCP sender compete for bandwidth?
  - When queues get to long: Packets loss
  - TCP sender decreases multiplicatively
  - UDP sender does not alter its sending rate
  - Resulting again in packet loss and decease of TCP bandwidth
  - ....

➔ Because of its congestion control the TCP sender will only get minimal bandwidth

➔ UDP is not “TCP friendly”

Summary

- Cost of congestion: Delay, packet loss, ...
- Congestion collapse
- TPC congestion control
  - Slowstart, congestion avoidance, fast retransmit, fast recovery
  - TCP Tahoe vs. TCP Reno
  - Additive increase / multiplicative decrease
- Fairness in TCP
  - Distributed convergence to fairness and efficiency
  - Requirements to ensure fairness
  - Fairness in situation TCP vs. UDP